

Điện toán biên trên nền Web cho Nhận dạng Dấu Quang học: Một đánh giá hiệu năng đa nền tảng

Huy Kien Phan^{*†}, Chau M. Truong^{*†}, Thanh Binh Nguyen^{*†}

^{*}Trường Đại học Công nghệ Thông tin, Thành phố Hồ Chí Minh, Việt Nam

[†]Đại học Quốc gia Thành phố Hồ Chí Minh, Việt Nam

22520709@gm.uit.edu.vn, {chautm,binhnt}@uit.edu.vn

Abstract—Nhận dạng Dấu Quang học (OMR) trên nền trình duyệt cho phép chấm điểm ngay trên thiết bị mà không cần cài đặt ứng dụng, nhưng chi phí tương đối của các giai đoạn khác nhau trong một pipeline OMR phía trình duyệt vẫn chưa được định lượng một cách hệ thống. Chúng tôi trình bày một khung (framework) Web Edge OMR cùng với phân bổ chi phí theo từng giai đoạn (stage) đầu tiên được báo cáo cho một pipeline OMR chạy hoàn toàn phía client trong trình duyệt, trong đó thị giác máy tính WebAssembly (WASM) và suy luận YOLO trên WebGPU đều thực thi cục bộ ngay trên thiết bị. Chúng tôi đánh giá nó trên một benchmark mẫu cố định gồm 185 phiếu, qua năm cấu hình trình duyệt/thiết bị, kèm một baseline Native Android để tham chiếu. Đánh giá của chúng tôi đưa ra ba phát hiện. Thứ nhất, khoảng cách độ trễ web-so-với-native bị chi phối bởi lỗi thị giác máy tính WASM có tính bất định. Lỗi này chạy chậm hơn $2.6\times$ so với phần tương ứng trên Native dưới cùng ranh giới giai đoạn, và vẫn nằm trên đường tới hạn (critical path) ngay cả khi việc định vị được chuyển sang bộ phát hiện, khiến nó trở thành mục tiêu tối ưu hàng đầu cho Web Edge OMR. Thứ hai, bất chấp khoảng cách này, cả hai cấu hình Web đều vượt 99% độ chính xác theo câu hỏi, và nhánh dấu góc đạt cùng mức đó khi được chuyển sang Native Android. Thứ ba, việc thêm một bộ phát hiện dấu góc trên WebGPU chỉ mang lại mức tăng độ chính xác nhỏ (~ 0.4 điểm), nên đầu ra của bộ phát hiện phải được kiểm chứng đầu-cuối (end-to-end) dựa trên bộ nhận dạng mẫu cố định, thay vì suy ra từ chất lượng định vị. Nhánh dấu góc đạt 99.46% tổng thể và 99.3% trên 101 phiếu trả lời của học sinh chưa từng xuất hiện trong quá trình phát triển bộ phát hiện. Những kết quả này ủng hộ việc chấm điểm trên nền trình duyệt như một lựa chọn không cần cài đặt, bảo vệ quyền riêng tư cho OMR trên thiết bị.

Index Terms—Nhận dạng dấu quang học, điện toán biên, trình duyệt web, YOLO, WebAssembly, WebGPU, benchmark đa nền tảng

I. GIỚI THIỆU

Chấm điểm tự động các phiếu trắc nghiệm được sử dụng rộng rãi trong các cơ sở giáo dục, nơi khối lượng lớn phiếu phải được xử lý chính xác trong thời gian ngắn. Các phương án triển khai hiện có đi theo hai hướng đã được thiết lập. Các dịch vụ dựa trên máy chủ truyền các phiếu được quét hoặc chụp về hạ tầng từ xa, gây phụ thuộc mạng và để lộ dữ liệu học sinh ra các hệ thống bên ngoài [1], [2]. Ứng dụng di động native tránh được việc truyền dữ liệu nhưng đòi hỏi đóng gói riêng cho từng nền tảng, phân phối qua kho ứng dụng, và chi phí bảo trì theo từng nền tảng.

Sự trưởng thành gần đây của WASM và WebGPU nay cho phép các khối lượng công việc thị giác máy tính và học sâu thực thi trực tiếp bên trong các trình duyệt hiện đại mà không cần plugin hay cài đặt, qua đó mở ra một mô hình triển khai thứ ba: một pipeline hoàn toàn phía client, trong đó ảnh phiếu trả lời được xử lý ngay trong trình duyệt và không bao giờ rời khỏi thiết bị. Việc thực thi cục bộ do đó tránh được rủi ro lộ dữ liệu vốn có ở cách chấm điểm phía máy chủ, trong khi một gói (bundle) triển khai qua một URL duy nhất phục vụ cả trình duyệt trên desktop lẫn di động giúp loại bỏ rào cản cài đặt của một ứng dụng native.

Mô hình không đồng nhất này hứa hẹn, nhưng hiệu năng của nó vẫn chưa được hiểu rõ. Một pipeline OMR trên trình duyệt ghép hai nền tảng thực thi rất khác nhau: thị giác máy tính WASM bị giới hạn bởi CPU và suy luận nơ-ron được tăng tốc bởi GPU, mỗi bên dùng các API và phần cứng trình duyệt riêng biệt. Các công trình hiện có chưa cho thấy hai nền tảng này tương tác ra sao khi cùng chạy trong một pipeline nhận dạng hoàn chỉnh. Vẫn chưa rõ điểm nghẽn chủ đạo nằm ở đâu, liệu việc định vị bằng WebGPU có đủ bù lại bất lợi về độ trễ của trình duyệt so với thực thi native hay không, và liệu một bộ định vị học máy có cải thiện nhận dạng đủ nhiều để biện minh cho việc thay thế front-end cổ điển hay không. Bởi vì chính chi phí tương đối giữa chúng, chứ không phải từng thành phần riêng lẻ, mới quyết định một cấu hình có sánh được với triển khai native hay không, nên cả hai phải được đo trong cùng một pipeline.

Để giải quyết điều này, chúng tôi trình bày một nghiên cứu thực nghiệm có hệ thống, đánh giá một pipeline Web Edge OMR hoàn chỉnh, ở cả cấu hình chỉ-CV lẫn cấu hình lai YOLO+CV, đối chiếu với một baseline Native Android. Phân tích của chúng tôi cho thấy rằng mặc dù WebGPU biến suy luận YOLO thành một bộ tăng tốc hiệu quả, lỗi thị giác máy tính WASM có tính bất định mới là nguồn lớn hơn của khoảng cách độ trễ web-so-với-native, và do đó là mục tiêu tối ưu hàng đầu cho OMR phía trình duyệt.

Bài báo có ba đóng góp:

- 1) Chúng tôi xây dựng và đánh giá một benchmark OMR mẫu cố định gồm 185 phiếu, được con người rà soát, qua năm cấu hình trình duyệt/thiết bị, với các phép đo có giám sát

về độ chính xác và độ trễ, các phép đo tài nguyên mang tính tham khảo, và một baseline Native Android trên hai thiết bị.

- 2) Chúng tôi thực hiện một phân tích phân bổ theo cấp độ giai đoạn (stage), tách bạch chi phí của thị giác máy tính trên trình duyệt và chi phí suy luận nơ-ron, chứng minh rằng khoảng cách độ trễ web-so-với-native lớn hơn ở giai đoạn thị giác máy tính WASM tất định so với ở suy luận YOLO được tăng tốc bởi WebGPU.
- 3) Chúng tôi đánh giá một cơ chế bàn giao (handoff) đầu góc YOLO chuyển việc định vị sang WebGPU trong khi vẫn giữ phần nhận dạng trong lõi CV mẫu cố định dùng chung, và nhận thấy một mức tăng độ chính xác nhỏ nhưng đo được, tập trung ở các ảnh chụp có độ tương phản thấp.

II. CÔNG TRÌNH LIÊN QUAN

a) *OMR cổ điển*: Các pipeline OMR truyền thống dựa vào các kỹ thuật xử lý ảnh thủ công như lấy ngưỡng, phát hiện biên, và các phép toán hình thái (morphological) để đăng ký (register) mỗi phiếu vào một mẫu cố định và đọc các ô tô ở những vị trí lưới định trước [1], [2]. Tuy hiệu quả về mặt tính toán, các phương pháp này vẫn nhạy cảm với biến thiên ánh sáng và méo hình học trong các tình huống chụp bằng thiết bị di động [3].

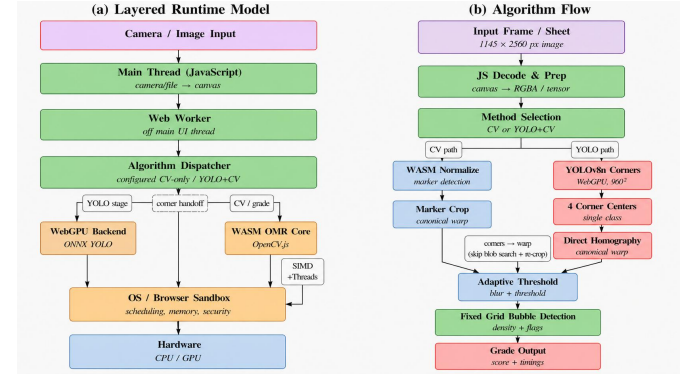
b) *Học sâu cho OMR*: Các nghiên cứu gần đây áp dụng các bộ phát hiện dựa trên YOLO [4] để cải thiện độ bền vững trong các điều kiện chụp ảnh khó. Ali và cộng sự [7] tổng hợp các bằng chứng cho thấy các bộ phát hiện dựa trên YOLO cải thiện độ bền vững phát hiện so với các phương pháp dựa trên ngưỡng, trong khi YOLOv8n [5] cung cấp một kiến trúc nhẹ phù hợp cho triển khai biên. Tabassum và Rahman [6] triển khai YOLOv8 cho OMR di động nhưng không báo cáo cả phân rã độ trễ theo giai đoạn lẫn một so sánh đa nền tảng có kiểm soát, khiến chưa rõ chi phí suy luận liên hệ thế nào với pipeline CV bao quanh trên các thiết bị bị giới hạn.

c) *Điện toán biên trên nền trình duyệt*: WASM và WebGPU [12] cùng nhau cho phép thực thi phía client trong các trình duyệt hiện đại, với suy luận nơ-ron được phơi bày qua ONNX Runtime Web [8]. Bằng chứng định lượng về chi phí phụ trội liên quan đã có sẵn: Jangda và cộng sự [9] báo cáo rằng WASM chạy chậm hơn trung bình 45% so với mã native trên các benchmark nặng về tính toán, một mức chênh có liên quan trực tiếp đến các khối lượng xử lý ảnh C++ được biên dịch sang WASM; còn Wang và cộng sự [10] đo được suy luận học sâu trên trình duyệt chậm hơn native $16.9\times$ trên CPU và $4.9\times$ trên GPU, với WebGPU nổi lên là back-end mạnh nhất phía trình duyệt. Tuy nhiên, các nghiên cứu này đặc tả các kernel riêng lẻ chứ không phải các pipeline thị giác máy tính hoàn chỉnh. Một mức trừu tượng cao hơn, API WebNN của W3C [11], cũng đang tiến triển qua quy trình chuẩn hóa của W3C, nhưng các pipeline phía client thực tế hiện nay vẫn thường dựa vào WASM và WebGPU, mà sự tương tác giữa chúng trong một ứng dụng đầy đủ thì còn ít được hiểu rõ.

Như vậy, các công trình trước hoặc xét độ chính xác OMR hoặc xét hiệu năng suy luận trên trình duyệt, nhưng hiếm khi cả hai trong cùng một hệ thống: các nghiên cứu OMR bỏ qua

chi phí các giai đoạn phía trình duyệt, trong khi các nghiên cứu hiệu năng trình duyệt chỉ đo các kernel riêng lẻ [9], [10] chứ không phải một pipeline nhận dạng hoàn chỉnh. Bài báo này lấp khoảng trống đó bằng cách lập hồ sơ (profiling) một pipeline OMR hoàn toàn phía client theo từng giai đoạn, qua nhiều back-end và cấu hình triển khai.

III. KIẾN TRÚC HỆ THỐNG VÀ PHƯƠNG PHÁP



Hình 1. Kiến trúc Web-Based Edge OMR. (a) Runtime phân tầng: luồng JavaScript chính điều phối đầu vào từ camera hoặc tệp tới một Web Worker. (b) Pipeline theo từng phiếu: WebGPU chỉ tăng tốc việc định vị YOLO, trong khi chuẩn hóa, lấy ngưỡng, đọc ô tô và chấm điểm vẫn nằm trong lõi WASM C++/OpenCV.

A. Kiến trúc phân tầng đa nền tảng

Hệ thống được tổ chức thành runtime phân tầng như trong Hình 1(a), nơi mỗi tầng che giấu tầng bên dưới sao cho cùng một mã ứng dụng chạy không đổi trên phần cứng không đồng nhất. Ở trên cùng, luồng JavaScript chính nắm việc tương tác với người dùng và thu nhận ảnh từ luồng camera hoặc tệp được tải lên, cùng việc vẽ chúng lên một canvas, nhưng nó ủy thác toàn bộ phần tính toán cho một Web Worker chuyên trách để việc xử lý nặng không bao giờ làm nghẽn khả năng phản hồi của giao diện. Bên trong worker, một Algorithm Dispatcher được cấu hình để chạy hoặc một pipeline chi-CV thông thường hoặc một pipeline lai YOLO+CV nhằm xác định các góc của phiếu; trong trường hợp lai, nó vận hành một mô hình YOLO ONNX trên một backend WebGPU để định vị, trong khi lõi OMR WASM (các kernel xử lý ảnh C++ viết tay cùng với OpenCV, được biên dịch sang WASM có dùng SIMD và đa luồng) thực hiện phần nhận dạng và chấm điểm thực sự. Vì dispatcher đưa mọi đầu ra định vị vào chính lõi WASM đó, hai pipeline vẫn hoán đổi được cho nhau ở giai đoạn nhận dạng. Cả hai back-end trình duyệt đều thực thi bên trong sandbox của trình duyệt, vốn điều phối lập lịch, bộ nhớ và quyền truy cập bộ tăng tốc trước khi ánh xạ công việc xuống CPU và GPU sẵn có. Sự tách biệt này cho phép một codebase duy nhất nhắm tới cả nền tảng desktop lẫn di động mà không cần chỉnh sửa.

B. Các pipeline thuật toán và cơ chế bàn giao

Hình 1(b) lần theo một phiếu từ lúc thu ảnh tới khi ra điểm đọc theo một trục chủ yếu tuyến tính, gồm Input Frame, JS Decode & Prep, Method Selection, tiếp đến là các nhánh phát

hiện CV hoặc YOLO song song, một Adaptive Threshold dùng chung, Fixed Grid Bubble Detection, và Grade Output. Phát hiện là giai đoạn duy nhất rẽ nhánh: nó tách thành một nhánh CV và một nhánh YOLO, cả hai lại hội tụ trước khi chấm điểm, sao cho phần còn lại của pipeline là như nhau ở cả hai cấu hình.

Luồng chính trước hết trao cho worker một khung hình thô hoặc ảnh được tải lên, ở đây là $1,145 \times 2,560$ px. *JS Decode & Prep* vẽ ảnh này lên một canvas và tạo ra hai sản phẩm mà các giai đoạn sau cần, cụ thể là một bộ đệm điểm ảnh RGBA cho lõi WASM và, mỗi khi nhánh lại được chọn, một tensor đã letterbox cho bộ phát hiện YOLO (960×960 đối với mô hình dấu góc chính). Hai nhánh tương ứng với các cấu hình triển khai được chọn một cách tường minh (benchmark đánh giá riêng từng cấu hình) chứ không phải chọn theo từng phiếu; trong nhánh lại, đường biến đổi từ góc (corner-warp) được dùng khi bộ phát hiện cho ra các góc hợp lệ, và nếu không thì lùi về bộ phát hiện cổ điển. Vì mỗi nhánh chỉ làm một việc duy nhất là đặt lưới đáp án vào các tọa độ đã biết và cả hai sau đó đều đưa vào cùng một bộ nhận dạng, nhánh được chọn chỉ thay đổi cách định vị phiếu, chứ không bao giờ thay đổi cách một đáp án tốt cuộc được đọc ra.

Trên nhánh CV, lõi WASM khôi phục hình học của trang mà không cần dẫn dắt bên ngoài. Giai đoạn *WASM Normalize* lấy ngưỡng ảnh, trích các thành phần liên thông (connected components), và giữ lại bốn dấu góc đăng ký bằng cách lọc các blob ứng viên theo diện tích, tỉ lệ khung và độ lấp đầy; khi tuyến dấu góc tỏ ra không đáng tin, nó thay vào đó lùi về biên giấy trắng. Từ bốn điểm tham chiếu thu được, nó ước lượng một phép biến đổi homography lên một mẫu chuẩn 1700×2400 , mà giai đoạn *Marker Crop* áp dụng để đưa lưới đáp án về các vị trí cố định.

Trên nhánh YOLO, cấu hình chính là một bộ phát hiện dấu góc: một bộ phát hiện *đối tượng* YOLOv8n đơn lớp (không phải đầu pose/keypoint) định vị bốn dấu góc in sẵn và dùng tâm các hộp bao (bounding box) của chúng làm các điểm đăng ký. Ảnh thô ($1,145 \times 2,560$ px) trước tiên được letterbox thành một tensor 960×960 bằng cách co theo chiều dài nhất và đệm 0 ở trục ngắn hơn, rồi mô hình chạy qua backend WebGPU. Sau khi suy luận, bốn hộp bao có độ tin cậy cao nhất được rút gọn về tâm của chúng trong hệ tọa độ tensor. Để quay lại pipeline dùng chung, các tọa độ này được ánh xạ ngược về không gian ảnh gốc bằng cách trừ đi các phần đệm và áp dụng hệ số co nghịch đảo. Các tâm đã khôi phục sau đó được sắp thứ tự theo hình học (trên-trái, trên-phải, dưới-phải, dưới-trái) và đưa *thẳng* vào phép homography để nắn phiếu lên mẫu chuẩn 1700×2400 . Trên nhánh này, các tâm góc được đưa thẳng vào hàm biến đổi, nhờ đó lõi WASM bỏ qua được cả việc phát hiện blob bằng thành phần liên thông của giai đoạn CV *WASM Normalize* lẫn lần dò lại dấu góc thứ hai, nên nhánh YOLO chỉ nhập lại bộ nhận dạng dùng chung tại giai đoạn *Adaptive Threshold*. Do đó YOLO chỉ đóng vai trò một front-end hình học; chuỗi đáp án luôn được giải mã bởi bộ nhận dạng CV dùng chung, cho phép đánh giá cơ chế bản giao mà không thay đổi phần chấm điểm phía sau.

Một khi hai nhánh đã nhập lại trên một ảnh đã nắn (rectified

crop) chung, *Adaptive Threshold* thực hiện làm mờ cục bộ và lấy ngưỡng, còn *Fixed Grid Bubble Detection* đọc các đáp án. Vì phiếu nay đã nằm trong tọa độ mẫu, lưới cố định trước tâm của mọi ô tô, nhờ đó mỗi lựa chọn có thể được chấm theo mật độ tiền cảnh (foreground) trong một cửa sổ lấy mẫu hình tròn, và các điểm số có thể được chuẩn hóa trong từng hàng, qua đó tránh việc một bản chụp tối đều hay mờ nhạt làm lệch quyết định. Giai đoạn cuối *Grade Output* xuất ra chuỗi đáp án đã giải mã cùng với các cờ hợp lệ và cờ dấu khả nghi, và các bộ đếm thời gian theo từng giai đoạn được báo cáo xuyên suốt bài báo này.

Bởi vì bộ nhận dạng có tính tất định, bất kỳ sai số hình học nào trước phép homography đều làm dịch toàn bộ lưới. Việc cắt ảnh có dẫn dắt bởi bộ phát hiện là một phần không thể tách rời của thuật toán nhận dạng, chứ không phải một bước tiền xử lý vô hại.

C. Tối ưu theo nền tảng và triển khai biên

Hai đích triển khai có những đánh đổi hỗ trợ cho nhau. Bản dựng Web Edge chạy không cần chỉnh sửa trong sandbox của trình duyệt với mức cài đặt bằng không, nhưng phải chấp nhận chi phí phụ trội do ảo hóa và chỉ có quyền truy cập bộ tăng tốc dưới sự quản lý. Bản dựng Native Android thì có quyền truy cập bộ tăng tốc trực tiếp và độ trễ suy luận thấp hơn rõ rệt, đổi lại là chi phí đóng gói riêng theo nền tảng và bảo trì. Vì cả hai đích đều thực thi mọi giai đoạn nhận dạng cục bộ trên thiết bị, ảnh phiếu trả lời không cần rời khỏi thiết bị. Độ trễ chỉ so sánh được giữa hai đích tại một ranh giới đo lường tương ứng, điều mà phần tiếp theo sẽ định nghĩa.

IV. THIẾT LẬP THỰC NGHIỆM

A. Tập dữ liệu và nhãn chuẩn

Tập đánh giá ($N=185$) gồm mọi phiếu đã được gán nhãn trong kho dữ liệu: 179 ảnh phiếu trả lời của học sinh và sáu phiếu đáp án, rút từ năm tập con (50, 48, 36, 46 và 5 phiếu; *dataset_3* chứa các ảnh chụp độ tương phản thấp, bị nén lại) và được chụp bằng thiết bị di động (danh nghĩa $1,145 \times 2,560$ px đối với *dataset_1*; các tập con khác có thể khác). *Dataset 3* là một tập con bị bạc màu, nén lại: cả 36 phiếu có dải tông bị nén (trung vị $p_{99} - p_1 \approx 129$ mức xám và màu trắng giấy bị giới hạn ở $\approx 192/255$, so với $\approx 225-237$ ở các tập con sạch nhất), khiến các dấu góc in sẵn nằm sát ngưỡng phát hiện bằng thành phần liên thông. Phiếu trả lời và phiếu đáp án là cùng một mẫu in (phiếu đáp án chỉ mang sẵn các dấu đúng), nên cả hai đều được nhận dạng bởi cùng một pipeline, phiếu đáp án chỉ khác ở chỗ nó còn được dùng làm tham chiếu chấm điểm. Hai người rà soát đã độc lập thiết lập nhãn chuẩn, giải quyết các bất đồng bằng kiểm tra thủ công, và đánh dấu các câu hỏi đuôi không hợp lệ là không áp dụng, cho ra 9,810 nhãn câu hỏi hợp lệ và 49,050 quyết định nhị phân theo từng dấu, làm nền cho mọi con số độ chính xác trong bài báo này.

B. Cấu hình huấn luyện bộ phát hiện

Bộ định vị dấu góc là một mô hình YOLOv8 nano đơn lớp (*marker_corners*) được tinh chỉnh (fine-tune) từ *yolo8n.pt*

trên 85 phiếu được gán nhãn, chia thành 68 để huấn luyện và 17 để kiểm định. Việc huấn luyện chạy tối đa 100 epoch (dùng sớm, patience 30) với batch size 8 và đầu vào 960×960 , dùng bộ tối ưu được Ultralytics tự chọn (tốc độ học ban đầu 0.01, momentum 0.937, weight decay 0.0005); việc tăng cường dữ liệu (augmentation) được cố ý giữ ở mức tối thiểu: chỉ dịch chuyển nhỏ (0.1) và co giãn (0.2), tắt lật ngang/dọc, mosaic và mixup, vì bốn dấu góc mang một thứ tự hình học cố định. Sau khi hội tụ, mô hình được xuất sang một định dạng ONNX thống nhất dùng chung cho cả triển khai Web và Native, qua đó loại bỏ biến thiên do chuyển đổi giữa các framework; chỉ khác nhau ở execution provider, là WebGPU trong trình duyệt và CPU hoặc NNAPI trên Android. Suy luận dùng đầu vào letterbox 960×960 với ngưỡng tin cậy 0.05 (giữ lại bốn hộp có độ tin cậy cao nhất) và NMS IoU 0.45. Trên tập kiểm định 17 ảnh, bộ phát hiện đạt $mAP@0.5=0.995$ và recall 1.0 (khôi phục đủ bốn dấu góc trên mỗi phiếu). Bởi vì bộ phát hiện chỉ định vị các mốc in sẵn cố định và không bao giờ định vị nội dung được tô, và vì 101 trong số 179 phiếu trả lời của học sinh trong benchmark chưa từng được dùng trong quá trình phát triển bộ phát hiện, độ chính xác nhận dạng trên benchmark (Mục V-A) đặc tả bộ nhận dạng khi đã có các góc được định vị, tách bạch với tập kiểm định của bộ phát hiện này. Nhận dạng ở giai đoạn CV có tính tắt định và giống hệt nhau qua các lần chạy lặp lại và qua các máy Windows.

C. Các nền tảng và máy chủ benchmark

Benchmark phát lại cùng một tập 185 ảnh qua năm cấu hình trình duyệt/thiết bị và một baseline Native Android trên hai trong số đó:

- **Máy tham chiếu desktop:** Intel Core i5-14600K, NVIDIA RTX 4070 Ti SUPER, 32 GB RAM.
- **Laptop:** ASUS Vivobook (i5-13500H, 24 GB RAM, GPU tích hợp Intel Iris Xe) và Acer Nitro 5 (i5-10300H, 16 GB RAM).
- **Android (Chrome/Blink):** máy tính bảng Poco Pad M1 (Snapdragon 7s Gen 4, 8 GB LPDDR4X) và Redmi Note 13 Pro+ (Dimensity 7200-Ultra, 8 GB LPDDR5).
- **Baseline native:** Poco Pad và Redmi còn chạy thêm một ứng dụng Android native qua các execution provider CPU đa luồng và NNAPI.

Web và Native dùng chung các giai đoạn thuật toán ở mức mã nguồn nhưng không ở dạng đã biên dịch: bản dựng OpenCV, bố cục bộ nhớ, lập lịch SIMD và thứ tự dấu phẩy động đều khác nhau giữa OpenCV.js và OpenCV Android, nên cùng một đầu vào không nhất thiết cho ra đầu ra giống hệt từng bit. Đích Native xuất các nhãn theo từng câu hỏi được căn theo nhãn chuẩn đã rà soát, nên nó được đánh giá cả về độ chính xác lẫn độ trễ. Để tái lập, bản dựng Web chạy ONNX Runtime Web trên WebGPU với một lỗi OpenCV biên dịch bằng Emscripten dùng SIMD và đa luồng dưới cô lập chéo nguồn (cross-origin isolation), còn bản dựng Native chạy ONNX Runtime Android với OpenCV 4; mỗi máy đo độ trễ được khởi động (warm up) bằng năm phiếu, và năm máy theo giao thức dấu góc được báo cáo theo trung vị của năm lần chạy.

D. Số đo và ranh giới đo lường

Để giữ cho các so sánh công bằng giữa môi trường sandbox và môi trường native, các số đo được giới hạn theo tầng trừu tượng runtime, và mỗi tuyên bố được gắn với sản phẩm và ranh giới của nó như sau. *Độ trễ Web* là độ trễ *pipeline phía worker*: thời gian thực thi C++ (và, với bản lai, WebGPU) phía worker, được đo sau khi dữ liệu ảnh tới được luồng worker, nên nó loại trừ phần giải mã và kết xuất ở luồng chính. *Độ trễ Native* là khoảng thời gian đầu-cuối dựa trên tệp, bao gồm giải mã và xử lý ảnh; thao tác ghi JPEG chỉ dùng để gỡ lỗi được loại khỏi các số đo báo cáo. Vì hai ranh giới khác nhau (Web loại trừ giải mã trong khi Native bao gồm nó), các so sánh độ trễ Web-Native trực tiếp chỉ được báo cáo trên các phép đo lỗi-giai-đoạn cùng ranh giới. *Mức sử dụng tài nguyên* được lấy mẫu bằng bất kỳ công cụ nào mà máy chủ phơi ra: các tiện ích tiến trình cho trình duyệt Windows và một bộ lấy mẫu trong-ứng-dụng/adb trên Android. Một tỉ số độ trễ cùng ranh giới dùng thời gian lỗi-giai-đoạn tương ứng (C++ phía worker trên Web và lỗi C++ native trên Android). Các tỉ số suy luận so thời gian ORT/WebGPU trên Web với thời gian ORT/NNAPI trên Android. Ở những chỗ bàn về độ trễ triển khai, ranh giới được nêu rõ, và thời gian tường (wall-time) đầu-cuối đầy đủ của trình duyệt, bao gồm cả giải mã và kết xuất ở luồng chính, không được xuất ra bởi các log hiện tại.

V. KẾT QUẢ VÀ THẢO LUẬN

A. Độ chính xác nhận dạng

Bảng I
ĐỘ CHÍNH XÁC NHẬN DẠNG THEO CÂU HỎI, CÓ GIÁM SÁT, TRÊN TẬP $N=185$ ĐÃ RÀ SOÁT.

Phương pháp	ĐCX câu	Câu đúng/GT	Phiếu đúng hẳn	GT trống	Dự đoán trống	Dự đoán đa
YOLO dấu góc	99.46%	9,757/9,810	155/185	44	23	13
CV truyền thống	99.03%	9,715/9,810	143/185	44	42	18

Dấu góc là bộ phát hiện 4 góc YOLO trong trình duyệt đưa thẳng vào một phép homography; cả hai dòng đều được đánh giá trên cùng 185 phiếu. *GT trống* đếm số câu hỏi hợp lệ mà đáp án đã rà soát để trống (các câu đuôi không hợp lệ bị loại khỏi 9,810); *Dự đoán trống/đa* đếm số câu mà phương pháp trả về là trống hoặc tô nhiều ô. P/R/F1 ở mức ô tô là 0.992/0.994/0.993 (CV) và 0.995/0.998/0.997 (dấu góc) trên 49,050 quyết định nhị phân.

Bảng I báo cáo nhận dạng khớp-chính-xác có giám sát trên tập mẫu cố định $N=185$ đã rà soát. CV truyền thống nhận dạng đúng 9,715 trong số 9,810 câu hỏi, tức độ chính xác theo câu hỏi là 99.03%, và tái tạo đúng mọi đáp án trên 143 trong số 185 phiếu. Thay giai đoạn định vị phiếu bằng CV bằng một bộ phát hiện dấu góc YOLO, vốn phát ra bốn dấu góc đăng ký và đưa thẳng tâm của chúng vào phép homography, cho ra 99.46% (9,757/9,810) và 155 trong số 185 phiếu đúng hoàn toàn, có thể tái lập qua năm máy theo giao thức dấu góc. Trên toàn benchmark 185 phiếu, bộ phát hiện trả về bốn dấu góc hợp lệ trên mọi phiếu, nên cấu hình lại không bao giờ phải gọi tới phương án dự phòng cố điển. Cần tách bạch hai đại lượng ở đây. *Độ chính xác định vị* được đo trên tập tách riêng 17 phiếu, nơi bộ phát hiện đạt $mAP@0.5=0.995$ và khôi phục đủ bốn góc trên mỗi phiếu. Còn con số 99.46% ở trên là một con số *nhận dạng đầu-cuối*: nó định lượng việc lỗi C++ tắt định đọc các ô tô tốt ra sao khi đã được cấp hình

học góc chính xác. Bởi vì các nhãn góc bao phủ một phần các phiếu đánh giá, đây là một cận trên tập đóng (closed-set), mẫu cố định cho nhận dạng khi đã có định vị tốt, chứ không phải một thước đo khả năng tổng quát hóa tập mở (open-set) của bộ phát hiện. Trên 101 phiếu trả lời của học sinh chưa từng xuất hiện trong quá trình phát triển bộ phát hiện, nhánh đầu góc đạt 99.3% độ chính xác theo câu hỏi (5,503/5,540), nằm trong sai số so với 99.6% trên 78 phiếu có chồng lấp với quá trình phát triển bộ phát hiện, cho thấy con số trên toàn tập không bị thổi phồng đáng kể bởi phần chồng lấp huấn luyện.

Một khoảng tin cậy 95% theo bootstrap-theo-phiếu là 98.63–99.37% cho CV và 99.20–99.68% cho nhánh đầu góc. Vì các câu hỏi trong cùng một phiếu có tương quan, chúng tôi kiểm định sự khác biệt bằng một bootstrap theo cặp trên 185 phiếu (10,000 lần lấy lại mẫu): mức tăng trung bình theo từng phiếu là +0.45 điểm (95% CI [+0.20, +0.72], không chứa số không), với nhánh đầu góc tốt hơn trên 32 phiếu và kém hơn trên 15 phiếu. Mức tăng tập trung ở các ảnh dataset_3 độ tương phản thấp, nơi việc tìm đầu góc bằng thành phần liên thông kém tin cậy nhất (CV 97.2% so với 99.6% ở đó). Xét theo số tuyệt đối, mức tăng này sửa đúng khoảng 42 câu trong số 9,810 (tập trung ở dataset_3 độ tương phản thấp), một biên độ không nhỏ khi yêu cầu chấm điểm chính xác ở mức từng phiếu. Do đó cải thiện này được hỗ trợ về mặt thống kê nhưng khiêm tốn về độ lớn, và vẫn nên được kiểm chứng đầu-cuối dựa trên bộ nhận dạng mẫu cố định thay vì suy ra chỉ từ chất lượng định vị.

Ngoài độ chính xác, việc đưa bốn điểm góc do bộ phát hiện cấp vào phép homography loại bỏ phần phát hiện đầu bằng thành phần liên thông cùng lượt dò lại thứ hai của nó khỏi lỗi C++ và chuyển việc định vị sang GPU; các kết quả độ trễ phía dưới cho thấy sự dịch chuyển kết quả về việc chi phí theo từng phiếu được tiêu tốn ở đâu. Chỉ riêng trên 179 phiếu của học sinh, CV đạt 99.00% và nhánh đầu góc 99.45% (cả hai đều $\geq 99.6\%$ trên sáu phiếu đáp án), nên các số đo không bị thổi phồng bởi các mẫu dễ.

B. Độ trễ đa nền tảng

Hình 2 và các Bảng II–III báo cáo benchmark triển khai, cho thấy khoảng cách Web-so-với-Native lớn hơn ở giai đoạn CV tất định so với ở suy luận YOLO.

Bảng chứng rõ nhất là một so sánh cùng-thiết-bị, cùng-ranh-giới trên Redmi. Với lõi CV (toàn bộ chuẩn hóa hai giai đoạn, cắt lại, lấy ngưỡng và đọc ô tô, do phía worker), Native Android cần 191 ms trong khi bản Mobile Web cần 503 ms, một khoảng cách 2.6× dưới cùng ranh giới giai đoạn, quy về ngăn xếp thực thi trình duyệt/WASM cùng với các khác biệt ở mức bản dựng như bố cục bộ nhớ, lập lịch SIMD và cách hiện thực OpenCV. Giai đoạn học máy thì hành xử rất khác: bộ phát hiện đầu góc mất 547 ms trên WebGPU so với 335 ms cho execution provider NNAPI của Native, một khoảng cách 1.6× nhỏ hơn nhiều so với mức chậm 4.9× phía GPU được báo cáo cho suy luận học sâu trên trình duyệt trong công trình trước [10], cho thấy WebGPU có thể là một bộ tăng tốc phía trình duyệt hiệu quả cho khối lượng công việc này. Vì khoảng cách CV vượt khoảng cách suy luận ở cả mức tương đối (2.6×

so với 1.6×) lẫn tuyệt đối (312 so với 212 ms), và vì một lỗi nhận dạng CV tất định vẫn nằm trên mọi phiếu ngay cả khi việc định vị được chuyển đi, đòn bẩy bậc nhất cho độ trễ Web Edge là runtime CV WASM, dù bộ phát hiện vẫn là một chi phí tuyệt đối đáng kể trên điện thoại và còn tăng thêm dưới các execution provider CPU (Bảng II).

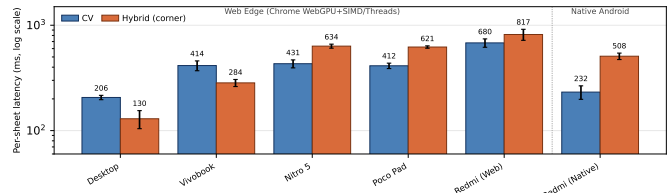
Ngân sách thời gian phía Native củng cố cho phân bổ này (Bảng II). Khi toàn bộ pipeline CV được chuyển sang (chuẩn hóa hai giai đoạn, lượt cắt lại đầu góc Stage-2, lấy ngưỡng và đọc ô tô), lõi CV Native chạy trong 191 ms và nhận dạng đúng 97.75% số câu. Khoảng cách còn lại so với nhánh Web CV (99.03%) không phải là giới hạn của bộ nhận dạng lưới cố định mà là của khâu tiền xử lý đầu góc nhảy với bản dựng: nó gói gọn trong ba ảnh độ tương phản thấp, bị nén lại, nơi mức xám của đầu nằm sát ngưỡng phát hiện, nên việc thu nhỏ và lấy ngưỡng của OpenCV-Android lệch khỏi các kernel tùy chỉnh của bản dựng Web vài mức xám và loại bỏ một đầu. Nhánh đầu góc tránh được kiểu thất bại theo từng tập con này: khi chuyển sang Native nó đạt 99.44% trên cả hai thiết bị (Redmi và Poco Pad), ngang với bản lai Web (99.46%) và bền vững qua cả năm tập con, kể cả tập con độ tương phản thấp. Native giữ độ trễ thấp hơn trong khi nhánh đầu góc dùng chung mang lại độ chính xác tương đương, bền vững qua các nền tảng.

Bảng II
Nhận dạng và độ trễ trên Redmi Native theo cấu hình ($N=185$, trung vị ms). Độ chính xác được đánh giá dựa trên nhãn chuẩn đã rà soát.

Cấu hình	ĐCX câu	ORT	Pipeline	e2e
CV	97.75%	—	191	227
Corner NNAPI	99.44%	335	476	502
Corner CPU-4T	99.44%	530	713	750
Corner CPU-1T	99.44%	956	1132	1174

Pipeline là thời gian xử lý OMR native; với các dòng Corner nó bao gồm suy luận ORT cộng phần xử lý OMR sau suy luận. e2e cộng thêm giải mã JPEG (thao tác ghi JPEG chỉ để gỡ lỗi, không có trong sản phẩm, bị loại); ORT là thời gian suy luận ONNX Runtime đo được. Dòng CV dùng toàn bộ pipeline đã chuyển sang gồm cả lượt cắt lại Stage-2 (97.75%); nhánh đầu góc được đánh giá dựa trên cùng nhãn chuẩn đã rà soát và bền vững qua mọi tập con.

Bên trong trình duyệt, bản lai đầu góc tái cấu trúc lại cấu trúc chi phí chứ không chỉ cộng thêm vào (Bảng III, trung vị năm lần chạy). Việc đưa hình học bốn điểm do bộ phát hiện cấp vào phép homography cho phép lõi C++ bỏ qua cả việc



Hình 2. Độ trễ trung bình theo từng phiếu theo nền tảng và cấu hình ($N=185$, thang log). Các cột Web thể hiện thời gian tương phía worker, bao gồm suy luận đầu góc trên WebGPU; các cột Native thể hiện thời gian đầu-cuối dựa trên tệp. Các ranh giới này không phải là tỉ số tốc độ trực tiếp; các tỉ số cùng ranh giới được báo cáo trong Mục V-B. Trên Redmi Web, bản lai làm tăng thời gian tương phía worker thêm 20% (680 → 817 ms).

phát hiện blob lần lượt dò lại đầu góc thứ hai, gần như giảm một nửa độ trễ theo từng phiếu: 503 $\rightarrow$ 249 ms trên Redmi, 343 $\rightarrow$ 166 trên Poco Pad, và 325 $\rightarrow$ 138 trên Vivobook. Việc toàn bộ pipeline có nhanh hơn hay không thì tùy thiết bị: trên Vivobook phần tiết kiệm ở C++ lớn hơn phần suy luận WebGPU thêm vào (thời gian worker 414 $\rightarrow$ 284 ms), trong khi trên điện thoại phần suy luận đầu góc 450–550 ms chiếm ưu thế và thời gian worker tăng lên (Redmi 680 $\rightarrow$ 817 ms). Như vậy bản lai chuyển việc định vị ra khỏi lõi tắt định và sang GPU, đánh đổi độ trễ suy luận lấy một giai đoạn CV nhẹ hơn với một mức tăng độ chính xác nhỏ.

Bảng III
ĐỘ TRỄ LỖI C++ THEO TỪNG PHIẾU (MS), CHI-CV SO VỚI BẢN LAI ĐẦU GÓC, QUA NĂM CẤU HÌNH TRÌNH DUYỆT/THIẾT BỊ.

Thiết bị	Ngân xếp	CV	Lai
Desktop (t.chiều)	Win/Blink	170	49
Vivobook	Win/Blink	325	138
Nitro 5	Win/Blink	344	94
Poco Pad	Android/Blink	343	166
Redmi	Android/Blink	503	249

Thời gian lỗi C++ phía worker. Bản lai còn chạy thêm bộ phát hiện WebGPU (suy luận $\approx$ 90 ms ở desktop tới $\approx$ 550 ms ở điện thoại, đã gộp vào thời gian tường phía worker trong Hình 2). Năm máy theo giao thức đầu góc là trung vị năm lần chạy ($N=185$). Nhận dạng đồng nhất giữa các thiết bị (CV 99.03%, lai 99.46%).

C. Mức sử dụng tài nguyên hệ thống

Mức sử dụng tài nguyên vẫn trong tầm kiểm soát (Bảng IV), dù các giá trị CPU và bộ nhớ là đặc thù theo công cụ. Các dòng Windows báo cáo CPU% và RSS của tiến trình renderer, trong khi Android Web và Native dùng một bộ lấy mẫu adb/trong-ứng-dụng trên thang 800%. Định bộ nhớ Web dao động từ khoảng 1.0–1.3 GB trên Chrome Windows tới 2.3 GB trên Nitro 5. Trên Redmi, RAM Native lấy mẫu được cao hơn RAM Web lấy mẫu được, nhưng điều này phản ánh ranh giới đo lường: bộ lấy mẫu native báo cáo PSS toàn tiến trình, trong khi bộ lấy mẫu trình duyệt chỉ bắt được tiến trình renderer và có thể bỏ sót các cấp phát ở tiến trình GPU dưới WebGPU. Do đó chúng tôi coi các số đo này là bằng chứng khả thi mang tính tham khảo chứ không phải các tỉ số hiệu quả.

Bảng IV
TÓM TẮT TÀI NGUYÊN TRÊN TOÀN LẦN CHẠY LIÊN TỤC $N=185$. Ô CPU LÀ TRUNG BÌNH/ĐÌNH THEO ĐƠN VỊ GỐC CỦA CÔNG CỤ; Ô RAM LÀ ĐÌNH CV/LAI TÍNH BẰNG MB. CHỈ MANG TÍNH KHẢ THI THAM KHẢO; CÁC ĐƠN VỊ KHÔNG SO SÁNH ĐƯỢC GIỮA CÁC CÔNG CỤ.

Máy	CPU (CV)	CPU (Lai)	RAM	Công cụ
Desktop	169/226%	130/248%	1,139/1,343	psutil
Vivobook	156/250%	248/331%	1,093/1,035	psutil
Nitro 5	159/219%	67/117%	2,318/2,089	psutil
Poco Pad M1	126/200%	94/151%	223/213	sampler
Redmi Web	177/271%	93/213%	254/216	sampler
Native (Redmi)	360/538%	98/393%	510/622	sampler

Các máy Windows dùng psutil báo CPU% renderer và RSS-MB; Android Web và Native dùng một bộ lấy mẫu adb/trong-ứng-dụng báo CPU% trên thang 800 và PSS-MB. Các cột CV/Lai của Android-Web là chi-CV so với bản lai đầu góc; Native dùng execution provider NNAPI.

Một mức đọc CPU trình duyệt thấp hơn không hàm ý thông lượng cao hơn: worker WASM ở gần mức một lõi, trong khi OpenCV Native chuyển mức CPU lấy mẫu cao hơn thành độ trễ thấp hơn nhờ đa luồng native.

D. Các mối đe dọa đến tính hợp lệ

- Các công cụ đo CPU/RAM khác nhau giữa các nền tảng, nên các giá trị tài nguyên mang tính tham khảo chứ không phải các tỉ số hiệu quả so sánh được. Các bảng độ trễ cấp giai đoạn báo cáo trung vị ở trạng thái ổn định sau khi khởi động, trong khi các cột trong hình cấp triển khai báo cáo giá trị trung bình theo từng phiếu; việc tải tài nguyên/mô hình lúc khởi động nguội bị loại trừ.
- Sản phẩm độ chính xác có giám sát bao phủ một bộ sưu tập mẫu cố định 185 phiếu. Đây là phạm vi khối lượng công việc dự kiến chứ không phải một tuyên bố về OMR bố cục tùy ý; các điều kiện chụp rộng hơn vẫn còn là phần kiểm chứng tương lai.
- Web và Native dùng chung các giai đoạn ở mức mã nguồn nhưng khác nhau ở bản dựng OpenCV, thứ tự đầu phẩy động và bố cục bộ nhớ, nên đầu ra không giống hệt từng bit. Tuy vậy nhánh đầu góc chuyển sang với độ ngang bằng (99.44% so với 99.46%); khoảng cách còn lại của riêng nhánh CV truy về khâu tiền xử lý đầu góc nhảy với bản dựng (Mục V-B).
- Bộ phát hiện huấn luyện trên 85 phiếu (68/17); nhánh đầu góc giữ mức 99.3% trên 101 phiếu trả lời của học sinh chưa từng thấy trong benchmark, nên con số 99.46% trên toàn tập là một kết quả tập đóng, mẫu cố định. Đánh giá tách rời theo người viết và đa mẫu vẫn còn là phần việc tương lai; chúng tôi không tuyên bố khả năng tổng quát hóa ngoài bố cục này.
- Độ trễ Web trên các laptop bị giới hạn nhiệt nhảy cảm với trạng thái nguồn và nhiệt: trên Vivobook chúng tôi thấy độ trễ C++ theo từng phiếu chênh tới khoảng $\approx 2\times$ giữa các lần chạy (đầu ra giống hệt từng bit). Do đó các máy theo giao thức đầu góc được lấy trung vị năm lần chạy, và so sánh Web-so-với-Native dùng cùng một thiết bị Redmi.

VI. KẾT LUẬN

Benchmark này cho thấy Web Edge OMR có thể vừa chính xác vừa khả thi để triển khai cho chấm điểm mẫu cố định, nhưng độ trễ của nó được quyết định bởi cách các giai đoạn cân bằng với nhau chứ không phải chỉ bởi suy luận. Nhánh Web CV đạt 99.03% độ chính xác theo câu hỏi và bản lai đầu góc đạt 99.46% qua năm thiết bị, ngang mức 99.44% khi chuyển sang Native. Bộ phát hiện học máy chỉ thêm một mức tăng độ chính xác nhỏ nhưng được hỗ trợ về mặt thống kê, tập trung ở các ảnh chụp độ tương phản thấp. Về độ trễ, số hạng web-so-với-native chủ đạo là giai đoạn CV WASM: trên cùng máy Redmi, 503 ms của nó gấp $2.6\times$ lỗi native 191 ms, một khoảng cách lớn hơn so với $1.6\times$ giữa WebGPU và NNAPI cho cùng bộ phát hiện. Do đó Native vẫn là phương án triển khai nhanh hơn ở mức độ chính xác tương đương, trong khi

Web Edge đòi độ trễ đó lấy quyền riêng tư và khả năng tiếp cận không cần cài đặt. Việc thu hẹp khoảng cách chủ yếu đòi hỏi một runtime CV WASM gọn nhẹ hơn với khả năng truy cập tốt hơn của trình duyệt tới tăng tốc CV, trong khi các triển khai lai trên di động cũng sẽ hưởng lợi từ suy luận bộ phát hiện nhẹ hơn.

TÀI LIỆU THAM KHẢO

- [1] R. Patel, S. Sanghavi, D. Gupta, and M. S. Raval, "CheckIt: A low cost mobile OMR system," in *Proc. IEEE TENCON*, 2015, pp. 1–6.
- [2] U. K. Okorie, O. Ezenwoye, and T. H. Kim, "Robust image processing based real-time OMR system," in *Proc. IEEE CSNT*, 2022, pp. 1–6.
- [3] M. Afifi and K. F. Hussain, "The achievement of higher flexibility in multiple-choice-based tests using image classification techniques," *IJDAR*, vol. 22, no. 2, pp. 127–142, 2019.
- [4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, real-time object detection," in *Proc. IEEE CVPR*, 2016, pp. 779–788.
- [5] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [6] K. Tabassum and Z. Rahman, "Optical mark recognition with object detection and clustering," in *Proc. IEEE ICEEICT*, 2024, pp. 352–357.
- [7] M. Ali, S. Hammoud, and A. Esper, "Optical mark recognition techniques for multiple-choice tests: A comprehensive review," *IEEE Access*, vol. 13, pp. 156407–156423, 2025, doi: 10.1109/ACCESS.2025.3600106.
- [8] Microsoft, "ONNX Runtime Web," 2021. [Online]. Available: <https://onnxruntime.ai/>
- [9] A. Jangda, B. Powers, E. Berger, and A. Guha, "Not so fast: Analyzing the performance of WebAssembly vs. native code," in *Proc. USENIX ATC*, 2019, pp. 107–120.
- [10] Q. Wang *et al.*, "Anatomizing deep learning inference in web browsers," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 2, Art. no. 47, 2024, doi: 10.1145/3688843.
- [11] W3C, "Web Neural Network API," Candidate Recommendation, 2026. <https://www.w3.org/TR/webnn/>
- [12] W3C GPU for the Web Working Group, "WebGPU," Candidate Recommendation, 2025. <https://www.w3.org/TR/webgpu/>